



UNIVERSITY OF BUCHAREST

**FACULTY OF MATHEMATICS
AND COMPUTER SCIENCE**



COMPUTER SCIENCE PROGRAM

Bachelor's Thesis

INLINING COMPILER OPTIMIZATION THROUGH MACHINE LEARNING TECHNIQUES

Graduate

Ingrid-Adriana Corobană

Scientific Advisor

Prof. Dr. Cristian Rusu

Bucharest, 2026

Abstract

The research area of compiler optimization remains relevant in computer science, since software is expected to stay compact across different hardware environments while also being fast enough to keep up with industry needs. Even though compile-time inlining is a fundamental optimization, choosing which call sites should be inlined remains complicated. One local choice can affect the future call graph and, therefore, later optimization opportunities. As a result, inlining can increase or decrease the size of the generated intermediate representation and of the final binary.

In this work, the inlining problem is framed in an offline policy-learning setting. For the implementation, C and C++ source files are collected from a small DCMTK snapshot and a varied public-source supplement. Deterministic synthetic modules are also added to extend the dataset with controlled inlining cases. The files pass through the Clang frontend with inlining turned off, in order to generate LLVM IR modules. The information extracted from individual call sites is translated into feature vectors.

Multiple heuristics with a teacher role are used. Each module passes through this heuristic set in order to determine which one minimizes rewritten IR size best for that module. The selected heuristic's call-site choices become the dataset for a supervised learning strategy, in which student models learn to mimic their teachers' paths through behavior cloning. The students' predictions are then integrated back into LLVM IR through call-site-level rewrites. The reported run compiles 163 C/C++ sources, evaluates 150 LLVM modules with local call sites, and extracts 10394 call-site samples. On the held-out split, the selected-teacher reference reduces rewritten LLVM IR instruction count by 1.10% relative to never-inline, while the best held-out student reaches a 0.61% reduction. These results do not outperform LLVM's production -Oz pipeline, but they give insight into the possibility of treating compile-time inlining decisions as machine-learning policy problems, rather than only as fixed classical heuristic problems.

Rezumat

Optimizarea compilatoarelor rămâne o ramură de cercetare importantă în informatică, deoarece tehnologia evoluează într-o direcție în care aplicațiile trebuie să fie compacte, rapide și eficiente pe platforme hardware diferite, ținând în același timp pasul cu cerințele de performanță ale industriei. Deși decizia de inlining la compilare reprezintă o optimizare fundamentală, alegerea call site-urilor care trebuie înlocuite prin inlining rămâne complicată, deoarece o decizie locală poate schimba graful de apeluri viitor, precum și posibilitățile de optimizare pe care compilatorul le mai poate folosi ulterior. Aceasta poate duce la creșterea sau scăderea dimensiunii reprezentării intermediare generate și a dimensiunii binarului final.

În această lucrare, problema inlining-ului este formulată într-o manieră de învățare offline. Pentru implementare, fișiere C și C++ sunt colectate dintr-un snapshot DCMTK, din surse generate sintetic și dintr-un supliment variat de surse publice selectate după compilare și filtrare după relevanța pentru inlining. Fișierele trec prin frontend-ul Clang, cu inlining-ul dezactivat, pentru a obține module LLVM IR. Informațiile extrase la nivel de call site sunt transformate în vectori de trăsături care includ informații despre caller, callee, call site și graful de apeluri.

Sunt folosite multiple euristici cu rol de profesor. Fiecare modul este evaluat de fiecare astfel de euristică, cu scopul de a determina care dintre ele minimizează cel mai bine dimensiunea LLVM IR rescrisă pentru modulul respectiv. Alegerile euristicii selectate la nivel de call site devin un set de antrenare pentru o strategie de învățare supervizată, în care modelele student învață să imite traiectoriile profesorilor prin behavior cloning. Predicțiile studenților sunt ulterior integrate înapoi în LLVM IR prin rescrieri la nivel de call site. Rularea raportată compilează 163 de surse C/C++, evaluează 150 de module LLVM cu call site-uri locale și extrage 10394 de exemple la nivel de call site. Pe split-ul held-out, referința selected-teacher reduce numărul de instrucțiuni LLVM IR rescrise cu 1.10% față de never-inline, iar cel mai bun student ajunge la o reducere de 0.61%. Rezultatele nu depășesc pipeline-ul de producție LLVM -Oz, dar demonstrează un workflow reproductibil de învățare offline pentru experimente cu politici de inlining la nivel de call site.

Contents

1	Introduction	1
1.1	Research Questions and Contributions	2
2	Compiler Background	3
2.1	Compilation Pipeline	3
2.2	LLVM and LLVM IR	4
2.3	Static and Dynamic Measurements	4
3	Function Inlining	5
3.1	Cases Where Inlining Helps	5
3.2	Cases Where Inlining Hurts	6
4	Problem Formulation	7
5	Related Work	9
5.1	Classical and Empirical Views of Inlining	9
5.2	Autotuning and Pre-MLGO Compiler Learning	9
5.3	MLGO	10
5.4	Offline Imitation Learning and BC-Max	10
5.5	Positioning Against Prior Work	11
6	Methodology	12
6.1	Dataset Summary	13
6.2	Teacher-Role Heuristics	14
7	Implementation	16
7.1	Repository Structure and Execution	16
7.2	Rewrite Algorithm	17
8	Evaluation	19
9	Limitations	21
10	Conclusions	22
A	Additional Diagnostics	23
	Bibliography	25

Chapter 1

Introduction

Optimizing at compile time remains a current research topic, since compilers must keep large builds deterministic enough while still producing compact, fast, cross-platform code. As a result, many optimization decisions are still made through heuristics: generalized rules that estimate whether a transformation is worth applying. This thesis studies inlining as one concrete case where a local decision can affect later passes, target behavior, and code size, so the best choice is too situational to be captured well by one simple rule [1, 2].

Inlining represents a high-impact example of this problem. By substituting a call site with the instructions corresponding to the called function, it can remove call overhead and expose constant propagation, devirtualization, dead-code elimination, and other later optimizations. Nonetheless, inlining can also duplicate code and grow the final binary when the copied body is not useful enough. Production compilers therefore use cost models and thresholds to balance optimization opportunity against generated-code growth [1, 2, 3].

The purpose of this thesis is to analyze a smaller ML-guided alternative to those manual baselines and to investigate whether a teacher-student model can give useful results from static features and scenario-specialized teachers. The implemented prototype evaluates several heuristics with a teacher role per module, selects the best teacher trajectory under a rewritten-IR size objective, and trains student policies to imitate the selected inline/no-inline decisions. The setup follows the broad MLGO idea that compiler decisions can be driven by features from the current context, while using offline imitation learning instead of online reinforcement learning [4, 3, 5, 6].

Function inlining is a useful case study because it requires both local and global information. At call-site level, the compiler decides whether to replace the call with the instructions of the called function, or to keep the function call unchanged. At global level, this choice can change the program representation: the caller can grow, the callee can become removable, the future call graph can change, and later optimization opportunities may also change. The prototype reproduces this setting in a smaller environment: heuristics with a teacher role create inline/no-inline trajectories, the best teacher per module is chosen by rewritten LLVM IR size, and student classifiers learn the selected labels.

For evaluation, LLVM IR is modified using LLVM’s `opt` tool. The number of rewritten LLVM IR instructions becomes the primary metric used to assess the student policies, while native object `.text` size is reported as a secondary reference metric.

1.1 Research Questions and Contributions

ID	Research question	Thesis contribution
RQ1	Can inlining be framed as offline imitation learning from heuristics with a teacher role?	Formalizes inlining as a binary call-site policy problem with teacher-role heuristics, selected trajectories, and behavior cloning.
RQ2	Can BC-Max-style selection produce call-site labels under a code-size objective?	Evaluates 16 teachers per module and builds selected-teacher labels for supervised training.
RQ3	Can simple students imitate selected teachers from static features?	Implements the LLVM IR workflow for feature extraction, teacher design, behavior cloning, rewriting, size measurement, and student training.
RQ4	Do learned decisions reduce rewritten IR compared with never-inline?	Applies predictions through cloned callees, <code>alwaysinline</code> , LLVM <code>opt</code> , and held-out IR/native <code>.text</code> evaluation.
RQ5	What gaps remain relative to production ML-guided inlining?	Analyzes dataset scale, static rewrite approximation, teacher quality, and missing full LLVM inliner integration.

Table 1.1: Research questions and corresponding contributions.

Chapter 2

Compiler Background

2.1 Compilation Pipeline

Compilation transforms source code into representations that are easier for a machine to execute. This thesis focuses on LLVM IR, since choosing whether to inline affects the instruction count and can be inspected before backend-dependent machine choices are made. A simplified LLVM-based pipeline is:

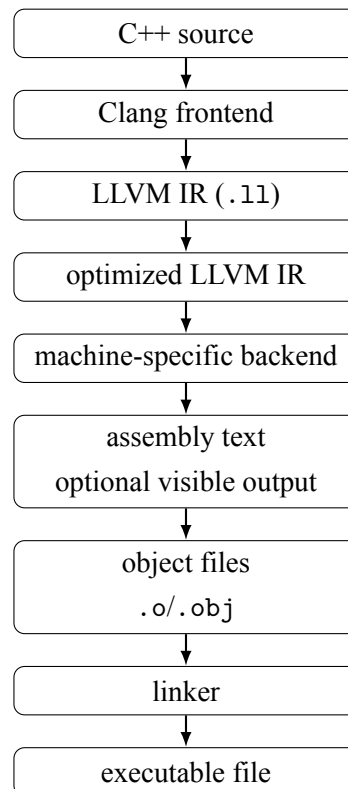


Figure 2.1: Simplified LLVM-based compilation pipeline.

Through Clang, C or C++ source files are checked according to the language rules and translated into LLVM IR. After IR optimization, the backend chooses target instructions, maps virtual registers to real registers, and emits object code [7, 8]. LLVM IR is therefore useful for this pro-

totype: it is lower-level than source code but still abstract enough to remain independent of the final object-file format.

2.2 LLVM and LLVM IR

LLVM is a compiler infrastructure project that includes optimizers, backend tools, and libraries. LLVM IR does not represent the entirety of the LLVM project, but only the intermediate representation used inside that infrastructure [9, 10]. It contains virtual registers, typed operations, branches, calls, and memory operations. These virtual registers are placeholders later lowered by the backend during instruction selection and register allocation.

2.3 Static and Dynamic Measurements

In this thesis, a distinction is made between static representations and dynamic executions. A static call graph contains possible calls before one concrete run is chosen, while a dynamic call graph keeps only calls that occur for one input. Likewise, static code size is measured in an artifact such as a native text section, while dynamic instruction count refers to executed instructions [7, 1].

Chapter 3

Function Inlining

Through inlining as a case study, various compiler optimization issues can be highlighted. Although it seems simple at first glance, inlining is important because one binary call-site decision can propagate through later compiler passes. It removes a call boundary by materializing the callee’s operations inside the caller. The original call instruction may disappear, the computational cost of dispatching through that call can be avoided, and later passes can optimize values and control flow across the merged code region [7, 8, 1].

3.1 Cases Where Inlining Helps

Small arithmetic helper functions represent typical good inlining candidates:

```
int ab(int a, int b) {                int sum_ab(int a, int b) {
    return a * b;                      return a + b + ab(a, b);
}
```

Figure 3.1: Small helper and caller before inlining.

Inlining `ab` removes a function call and replaces it with a small multiplication expression. If this call belongs to a hot part of the program, meaning a part that is executed frequently, saving the call overhead can be beneficial.

Inlining is also useful in cases where constants can propagate. In the example below, after inlining, the compiler can understand that `c` is always zero in this call context, and this can lead to the simplification of the other branches:

```

// before:
string light_color(int c, int i) {
    if (c == 0)
        return light_red(i);
    if (c == 1)
        return light_green(i);
    return light_blue(i);
}

string car_color(int i) {
    int f = i;
    return light_color(0, 2 + f);
}

// after:
string car_color(int i) {
    int f = i;
    return light_red(2 + f);
}

```

Figure 3.2: Constant-propagation example before and after inlining.

3.2 Cases Where Inlining Hurts

Recursive functions can lead to uncontrolled code expansion, and in such cases inlining should usually be avoided:

```

int sum(int x) {
    if (x <= 0)
        return 0;
    return x + sum(x - 1);
}

// recursive inlining:
// exposes another recursive call
// repeated expansion
// needs recursion/growth checks

```

Figure 3.3: Recursive call site and the reason it is usually rejected.

In contrast with hot parts of a program, choosing to inline a rarely called large function can be damaging:

```

void game_engine_reinit() {
    // large recovery path
}

void entry(bool error) {
    if (error)
        game_engine_reinit();
}

```

Figure 3.4: Cold-path callee and caller: inlining the recovery function can grow static code size and instruction-cache pressure with little hot-path benefit.

Another bad case appears when a large callee is copied at multiple call sites:

```

int get_info(int id) {
    int s = id;
    for (int i = 0; i < 1000; ++i)
        s = (s * 33) ^ i;
    return s;
}

void info_candidates() {
    cout << get_info(12) << endl;
    cout << get_info(133) << endl;
    cout << get_info(144) << endl;
    // can grow code unnecessarily
}

```

Figure 3.5: Large callee copied at multiple call sites if inlined aggressively.

Chapter 4

Problem Formulation

Let P be a program or module, and let

$$C(P) = \{c_1, c_2, \dots, c_n\}$$

be the set of all call sites observed during the inlining pass. Call sites are the locations of function calls, where a caller function invokes another function, called the callee. For each call site c_i , the compiler can extract information from the call site and from the call graph in order to form a feature vector:

$$x_i = \phi(c_i),$$

where ϕ is the feature-extraction function. In this thesis, the static features include caller and callee shape, instruction and call counts, constant-argument information, recursion information, and limited call-graph context.

The possible action choice is binary:

$$A = \{0, 1\}, \quad 0 = \text{do not inline}, \quad 1 = \text{inline}.$$

A policy therefore refers to a function

$$\pi_\theta : \mathcal{X} \rightarrow A, \quad \pi_\theta(a \mid x)$$

where the second form denotes the likelihood of choosing action a , given feature vector x .

Because of inlining, the problem is chronological and interdependent. If a policy π is applied to a program P , it creates the trajectory:

$$\tau_\pi(P) = ((x_1, a_1), (x_2, a_2), \dots, (x_T, a_T)),$$

where T is the number of inlining decisions made for that program. After one call site is inlined, the caller may grow, the callee may become dead, and later simplifications may appear. Local decisions are therefore not fully independent [3, 6].

The implementation uses a static offline approximation of this formulation: it collects call sites from the initial LLVM IR, predicts labels, and applies selected rewrites in one pass. It does not attempt online learning, where the model would keep learning from new feedback during compilation. It also does not update the call graph and recompute features after each individual rewrite, as LLVM’s online inliner could. Instead, the experiment trains from a fixed dataset and applies a fixed batch of decisions to each module.

The objective is to minimize generated code size. In the prototype, the main way of measuring how good a policy is uses the final rewritten LLVM IR instruction count:

$$\text{Size}_{IR}(P, \pi), \quad R(P, \pi) = -\text{Size}_{IR}(P, \pi).$$

A lower value is better; equivalently, the reward is written as negative size.

Let

$$\mathcal{H} = \{h_0, h_1, \dots, h_k\}, \quad h^*(P_j) = \arg \min_{h_i \in \mathcal{H}} \text{Size}_{IR}(P_j, h_i)$$

be the set of heuristics with a teacher role and the selected teacher for module P_j . Each heuristic h_i maps call-site-level information to inline/no-inline choices.

The trajectory corresponding to the selected teacher is then added to the supervised learning data:

$$\mathcal{D} = \bigcup_j \{(x_{j,t}, a_{j,t}^*)\}_{t=1}^{T_j},$$

where $a_{j,t}^*$ is the action taken by the selected teacher $h^*(P_j)$ for feature vector $x_{j,t}$.

A student model π_θ is trained through behavior cloning. In the ideal case, the student minimizes disagreement between its own choices and the actions chosen by the selected teacher:

$$\min_\theta \sum_{(x,a) \in \mathcal{D}} \mathbf{1}[\pi_\theta(x) \neq a], \quad \mathcal{L}(\theta) = - \sum_{(x,a) \in \mathcal{D}} \log \pi_\theta(a \mid x).$$

In practice, the implementation trains standard classifiers; for probabilistic models, the second expression is the binary cross-entropy.

After training, the student can be treated as a new heuristic. At each call site, it receives the current feature vector and predicts whether to inline:

$$\hat{a} = \arg \max_{a \in \{0,1\}} \pi_\theta(a \mid x).$$

The predicted decisions are then applied in the LLVM IR rewrite stage, which is necessary in order to obtain the final rewritten instruction count used as the metric.

Chapter 5

Related Work

5.1 Classical and Empirical Views of Inlining

Inlining has been studied long before modern learned compiler policies. A classical inliner estimates the benefit of removing a call and enabling later optimizations, then compares it with the risk of code growth. Theodoridis, Grosser, and Su show that optimal inlining remains nontrivial because decisions interact with call chains, code size, and later passes [2]. Godbolt’s compiler-optimization examples give the same practical motivation: inlining can expose constants, unreachable branches, and other caller-specific information hidden behind the call boundary [1, 11].

5.2 Autotuning and Pre-MLGO Compiler Learning

Before MLGO, several works already treated compiler optimization as empirical search or learning. MILEPOST GCC uses program features to select optimization choices instead of relying only on one fixed optimization level [4]. Its relevance here is methodological: it shows that compiler heuristics can be learned from empirical data, although it addresses broader optimization selection rather than call-site-level inlining.

Inlining-specific autotuning also predates the current MLGO line of work. Cavazos and O’Boyle study automatic tuning of inlining heuristics, showing that parameters of an inliner can be adjusted empirically instead of being chosen entirely by hand [12]. This is close to the problem in this thesis because it focuses on inlining, but the emphasis is different. The goal there is to tune a heuristic; the goal here is to learn a policy from selected teacher trajectories. In other words, autotuning searches for better heuristic parameters, while behavior cloning tries to learn a decision rule over call-site features.

More recently, MLGOPerf extends ML-guided inlining toward performance rather than only code size [13]. Runtime can be more directly useful for deployed software, but it introduces measurement noise and dependence on hardware and inputs. This thesis therefore uses LLVM IR

instruction count as its primary low-compute metric and reports native .text size as a secondary metric.

5.3 MLGO

MLGO represents a starting point and a primary example of how learned decision-making policies can be integrated into LLVM optimization heuristics [3, 5]. For inlining, the state is represented by current call-graph and call-site features, the action is inline or no-inline, and the reward is a final compiled-program improvement, often native code-size reduction for inlining-for-size. Because the full call graph is expensive to use directly, the compiler exposes a feature vector about the caller, callee, call site, and global graph. The original MLGO inlining setup uses policy-gradient training with a delayed reward, which is powerful but expensive because each environment interaction requires compilation [3].

5.4 Offline Imitation Learning and BC-Max

Marinov, Agarwal, and Trofin propose a cheaper alternative to online reinforcement learning through offline imitation learning from multiple baselines [6]. In the inlining setting, let

$$\mathcal{H} = \{h_0, h_1, \dots, h_n\}$$

be the set of heuristics with a teacher role, where P_i is one module and $\text{Size}_{IR}(P_i, h)$ is the rewritten LLVM IR instruction count obtained when teacher h is applied to P_i .

For each module, the best teacher is the one with the smallest rewritten IR size:

$$h_i^* = \arg \min_{h \in \mathcal{H}} \text{Size}_{IR}(P_i, h).$$

The selected-teacher trajectory is then:

$$\tau_i = \{(s_1, h_i^*(s_1)), \dots, (s_m, h_i^*(s_m))\}.$$

Here, s_t is the feature vector for the t -th call site. The union of these trajectories becomes a supervised dataset, and a student policy h_{n+1} learns selected-teacher labels rather than exploring by repeated compilation.

Teacher labels are not universal labels for isolated call sites. Rather, they represent what the selected teacher thinks is best in that setting. Since decisions are not only call-site-dependent, but also context-dependent, similar local call sites may receive different labels in different modules if the best module-level teacher is different. This creates possible label conflicts in the behavior-cloning dataset. It is also one reason why the final result must be measured after applying the predicted rewrites, not only by checking label agreement.

5.5 Positioning Against Prior Work

Table 5.1 summarizes how the prototype borrows the policy-learning view from MLGO and BC-Max while remaining smaller and more inspectable than production compiler infrastructure.

Work	Main idea	Evaluation focus	Relation to this thesis
Cavazos and O’Boyle [12]	empirical tuning of inliner parameters	benchmark performance	early evidence that inlining choices can be optimized empirically
MILEPOST GCC [4]	program-feature-based optimization selection	empirical benchmark performance	broad compiler ML rather than call-site-level inlining
MLGO [3, 5]	learned policies integrated inside LLVM	code size and performance on large corpora	main learned-heuristic reference, not reproduced here
BC-Max [6]	offline imitation learning from multiple baselines	final objective value	closest template for teacher selection and behavior cloning
Theodoridis et al. [2]	autotuned search for high-reduction inlining-for-size decisions	native binary size on SPEC2017, LLVM, and SQLite	motivates code-size evaluation and stronger baselines
This thesis	BC-Max-style supervised pipeline over static call-site features	rewritten IR and native text-section reference	inspectable offline pipeline for call-site-level policy experiments

Table 5.1: Related-work positioning for ML-guided and empirical inlining optimization.

Chapter 6

Methodology

The experimental pipeline follows Figure 2.1 until LLVM IR is produced, and the main goal is to output LLVM IR that is as small as possible. From that point, the prototype extracts call-site features, evaluates teachers, selects the best teacher per module, trains student classifiers, and applies predicted decisions through an opt-based IR rewrite. Source collection, IR generation, teacher choice, student training, rewriting, and measurement remain visible in the repository.

The C/C++ sources include a small DCMTK snapshot¹, 70 deterministic generated modules, and 76 public-source files selected after compilation and inlining-relevance filtering. The generator samples a $4^6 = 4096$ configuration space covering callee profiles, call mixes, argument patterns, helper-body shapes, driver shapes, and explicit size profiles. This creates tiny helpers, branch-heavy helpers, medium arithmetic helpers, large shared callees, recursive calls, variable operands, and constant operands. The 70 generated modules contain 1120 driver call sites, giving the teachers both inline and no-inline cases while broader benchmark transfer remains future work.

The public-source supplement is kept only when a file compiles to LLVM IR, exposes at least one local call site, and produces a measurable teacher-selection artifact through the rewrite pipeline. These filters are applied before student training and are part of dataset construction. The implemented features are a smaller local approximation of the public MLGO inlining schema: caller, callee, call-site, and module graph information, with instruction counts, call counts, constant arguments, and recursion indicators added for interpretability.

¹DCMTK source repository: <https://github.com/DCMTK/dcmtk>.

Feature group	Implemented fields	Type	What the values indicate
Caller shape	caller-prefixed counts: basic blocks, conditional blocks, users, IR instructions, and call instructions	numeric	size and branching of the function that receives the inlined body; large or call-heavy callers may already be expensive to grow
Callee shape	callee-prefixed counts: basic blocks, conditional blocks, users, IR instructions, and call instructions	numeric	estimated body copied by inlining, plus whether the callee has few users and may become removable after all useful calls are inlined
Call-site context	callsite_height, cost_estimate, and number of constant parameters	numeric	local decision context: approximate graph depth, proxy inline cost, and constants that may enable simplification after the callee body is copied into the caller
Module graph	module edge count and node count	numeric	rough module-level density, used to distinguish sparse files from call-heavy modules where one decision can interact with many nearby calls
Safety signal	recursive-call indicator	binary	marks recursive expansion patterns, where repeated inlining can grow code unnecessarily

Table 6.1: Grouped feature schema used by all teachers and students. The complete ordered feature list is fixed in `scripts/data.py`.

6.1 Dataset Summary

The current prototype uses a DCMTK-based source snapshot, generated modules, and a public-source supplement in order to test the offline imitation-learning pipeline on a nontrivial number of call sites.

Quantity	Value
Source mix	DCMTK, generated, 76 public files
Candidate C/C++ files	164
Compiled C/C++ files	163
Modules with local call sites	150
Extracted call sites	10394
Train samples	9009
Hold-out samples / modules	1385 / 26
Teacher policies / students	16 / 7
Primary metric	rewritten IR instructions
Secondary metric	native .text size

Source group	Teacher IR red.	Student IR red.
Generated	8.76%	2.75%
Real source	1.22%	1.08%
All modules	1.66%	1.18%

Table 6.2: Dataset summary and source-group IR reduction. “Teacher” means the selected-teacher reference; “Student” means the best full-set student conservative_forest. The most frequent selected teachers are small_callee on generated modules and greedy_ir_size on real-source modules.

At call-site level, 9128 of the 10394 examples come from public source files. The train/test split is performed by module rather than by individual call site, so all call sites from one module stay on the same side of the split. This avoids testing on contexts that are nearly duplicated from the same caller, callee, or call graph.

Candidate group	Files	Compiled	Included	Reason
DCMTK snapshot	18	17	yes	source-level local call sites
Generated modules	70	70	yes	controlled inlining and no-inlining patterns
Public sources	76	76	yes	compiled C/C++ files with local call-site signal

Table 6.3: Dataset accounting. Filtering is applied before student training and keeps source-level modules with local call sites relevant to the rewritten-IR objective.

6.2 Teacher-Role Heuristics

The local set of teacher-role heuristics is intentionally simple and inspectable. The teachers use deterministic rules starting from the same call-site-level feature vector. Let s be callee size, u callee users, b branch count, k constant arguments, and r a recursion flag. Most rules use the following general shape:

inline if $B(x) - C(x) > 0$, $B(x) \approx 2k + \mathbf{1}_{u=1} + \text{depth signal}$, $C(x) \approx s + b + \text{nested calls} + r$.

The exact thresholds are fixed in `teach.py`; the table gives the readable form of each rule. The exception is `greedy_ir_size`, whose labels are produced by measured rewrites in `rewrite.py`. Therefore, most teacher-role heuristics are cheap proxy policies, while the greedy heuristic is slower because it actually tests candidate rewrites.

Teacher	Short rule
<code>never_inline</code>	Conservative baseline: never inline.
<code>small_callee</code>	Inline if s is small and $r = 0$.
<code>single_caller</code>	Inline if $u = 1$ and the callee is not large; the standalone helper may disappear after cleanup.
<code>constant_argument</code>	Inline if $k > 0$, so caller-known constants may remove branches or simplify expressions.
<code>cost_budget</code> , <code>growth_budget</code>	Inline if estimated growth $G \approx s \max(1, u) + b$ + nested calls stays under a budget.
<code>loop_averse</code> , <code>hot_leaf</code>	Prefer leaf-like callees; reject large, branchy, or call-heavy callees.
<code>balanced_score</code> , <code>llvm_like_size</code> , <code>benefit_cost_ratio</code>	Inline if $B(x) / \max(1, C(x))$ passes a threshold.
<code>aggressive_speed</code>	Accept more small and medium calls, while still rejecting recursive or clearly large cases.
<code>ml_linear_score</code>	Inline if $w \cdot x + \beta > 0$, using fixed hand-written weights rather than training.
<code>bandit_ucb_proxy</code>	Inline if $\hat{g}(x) + \alpha / \sqrt{1 + u} > 0$: estimated gain plus a small uncertainty bonus.
<code>rl_value_proxy</code>	Inline if immediate simplification plus future call-graph value exceeds growth penalty.
<code>greedy_ir_size</code>	Measured teacher: order plausible non-recursive candidates, try candidate rewrites, and keep a candidate only when the actual rewritten IR instruction count decreases.

Table 6.4: Teacher policies as readable decision rules.

The ML-, bandit-, and RL-inspired teachers are approximations rather than actually trained teachers. The measured greedy teacher is more expensive because it orders plausible candidates from static features, then uses the same clone-and-alwaysinline rewrite path as evaluation and keeps only locally improving rewrites. This distinction matters because the student does not learn from a single oracle; it learns from the teacher that behaves best for each module under the rewritten-IR objective.

The main reported run uses one offline teacher-selection round: teachers are evaluated, one teacher trajectory is selected per module, and students imitate those selected labels. Chapter 8 also reports a small three-round check in which the best learned student is fed back as another measured candidate policy.

Chapter 7

Implementation

The main pipeline and implementation are located in the repository root. The implementation generates synthetic modules, uses compilation stages to reach LLVM IR, extracts call-site features, evaluates teacher policies, trains students from the best-performing teachers, rewrites selected call sites, and writes artifacts that can be inspected. The source repository is available at:

<https://github.com/dirgnic/optimizingInlining>

7.1 Repository Structure and Execution

The scripts cover generation, feature extraction, teacher selection, student training, rewriting, and figure generation. After cloning the repository, the full pipeline can be run from the repository root with `python3 scripts/run.py all`. A short inspection run is available as `python3 scripts/run.py demo`; it compiles a few generated modules, prints the extracted call-site order and inline/keep decisions, and writes before/after demo IR under `out/demo/rewritten/`. The reported numbers come from deterministic source generation, a small number of DCMTK files, and a fixed module-level train/test split, with artifacts written under `out/`.

The default execution path is organized as shown in Table 7.1. Only scripts used by the reported run are listed here.

Step	Script	Role
0	<code>data.py</code>	defines shared directories, feature names, and call-site data structures.
1	<code>run.py</code>	coordinates the full command <code>python3 scripts/run.py all</code> and the short demo command <code>python3 scripts/run.py demo</code> .
2	<code>build.py</code>	generates the deterministic synthetic modules, compiles C/C++ sources to LLVM IR, and measures native compiler baselines.
3	<code>ir.py</code>	parses LLVM IR and extracts feature vectors for local call sites.
4	<code>teach.py</code> and <code>rewrite.py</code>	apply teacher rules, rewrite IR with LLVM opt, and select the best teacher per module by rewritten IR instruction count.
5	<code>teach.py</code>	converts the selected teacher actions into the behavior-cloning dataset.
6	<code>learn.py</code>	trains the student classifiers and stores their inline/no-inline predictions.
7	<code>rewrite.py</code>	applies student predictions through the same clone-and-opt rewrite path, then measures rewritten IR and native text-section size.
8	<code>graph.py</code> , <code>paper.py</code> , and <code>visual.py</code>	write the call-graph trace, generated Markdown summary, and figures used for inspection and reporting.

Table 7.1: Default script order for the reported thesis pipeline.

The local run uses macOS on Apple Silicon, Homebrew Clang/LLVM 15.0.7, and Python 3.9.6. Source files are translated to LLVM IR using the Clang frontend with a strict no-inlining configuration, in order to observe the call-graph structure before premade inlining optimizations. Reproducibility comes from deterministic generation, fixed model seeds in `learn.py`, and a deterministic module-level split.

7.2 Rewrite Algorithm

The rewriting algorithm works at call-site level and applies an LLVM IR transformation. For each selected call site, the callee is cloned, the selected call is redirected to the clone, and LLVM performs inlining and cleanup. Recursive or unresolved cases are skipped. This step is not the actual LLVM inliner, but a smaller reproducible inlining mechanism built for the experiment. During evaluation, inline predictions trigger this rewrite path, while no-inline predictions leave the call site unchanged; the final metric is therefore measured after predictions are materialized in LLVM IR.

The call-site order is deterministic and textual. Modules are analyzed in sorted .11 file-

name order. Within each module, functions are visited in the order in which their LLVM define blocks appear. Within each function, call instructions are scanned from top to bottom. This is not dynamic execution order and not a topological call-graph order. Non-greedy teachers and student policies keep this extracted order when preparing rewrites. The measured greedy_ir_size teacher is the exception: it first filters plausible non-recursive candidates, orders them by benefit/cost score, and accepts a candidate only when the actual rewritten IR instruction count decreases.

Student model	Role in the experiment
logistic_regression	Linear, interpretable classifier.
decision_tree	Small threshold tree over individual features.
knn	Copies labels from nearby feature vectors.
random_forest	Ensemble of shallow trees.
deep_forest	Forest with deeper trees, giving a more flexible rule set.
conservative_forest	Forest variant with a higher effective inline threshold.
small_mlp	One-hidden-layer neural classifier.

Table 7.2: Student model families used for behavior cloning.

The student models do not discover useful inlining decisions by recompiling programs or exploring random actions during training. They receive the same static call-site feature vectors and learn, through behavior cloning, to minimize disagreement with the selected teacher-role heuristic for each module. Their quality therefore depends on both parts of the setup: whether the teacher set contains useful trajectories and whether the chosen model family can generalize those trajectories to held-out modules. This is why the table uses several simple classifiers rather than a single learner; each one represents a different compromise between interpretability, flexibility, and risk of copying noise from the selected labels.

Chapter 8

Evaluation

In the evaluation stage, 163 C/C++ sources are compiled, resulting in 150 LLVM modules with local call sites and 10394 call-site samples. The deterministic module split uses 9009 samples from 124 modules for training and 1385 samples from 26 modules for held-out testing. Teacher-label match measures agreement with selected-teacher labels; the actual code-size effect is measured only after predicted rewrites are applied.

In the teacher-evaluation and selection stage, every teacher-role heuristic is run on each module in order to determine which one produces the smallest rewritten LLVM IR. Across all 150 modules, average rewritten IR falls from 1985.39 instructions for never-inline to 1952.39 for the selected-teacher reference, which means a 1.66% reduction. On the held-out split, the corresponding reduction is 1.10%: 1569.23 average IR instructions for never-inline versus 1552.04 for the selected-teacher reference.

Model	Teacher-label match	Held-out Avg. IR	IR reduction	Gap to teacher
logistic_regression	0.901	1569.69	-0.03%	17.65
decision_tree	0.944	1565.50	0.24%	13.46
knn	0.914	1588.08	-1.20%	36.04
random_forest	0.952	1562.46	0.43%	10.42
deep_forest	0.955	1559.62	0.61%	7.58
conservative_forest	0.947	1562.69	0.42%	10.65
small_mlp	0.931	1607.35	-2.43%	55.31

Table 8.1: Held-out student results on the 26 test modules. Teacher-label match measures imitation of selected-teacher labels; IR reduction and gap to selected teacher measure the rewritten-IR result.

Teacher-label match is only a behavior-cloning diagnostic. The IR size is measured only after the predicted rewrites are applied and the instruction count is recomputed. `small_mlp` matches labels reasonably well but still grows IR, while `deep_forest` is closest to the selected-teacher reference in rewritten IR size. Similar label agreement can therefore lead to different size behavior.

Policy	Held-out Avg. IR	IR reduction
never_inline	1569.23	0.00%
attempted always-inline	1995.27	-27.15%
selected teacher	1552.04	1.10%
deep_forest	1559.62	0.61%
conserv. forest	1562.69	0.42%

Table 8.2: Held-out reference comparison.

Native workflow	.text bytes	Obj. bytes
no_inline_0z	674660	2225096
llvm_0z	574044	1585120
Reduction	14.91%	28.76%

Table 8.3: Compact LLVM -0z native-size comparison.

Scope	Teacher IR red.	Student IR red.
All 80	1.22%	1.08%
13 held-out	0.32%	0.19%

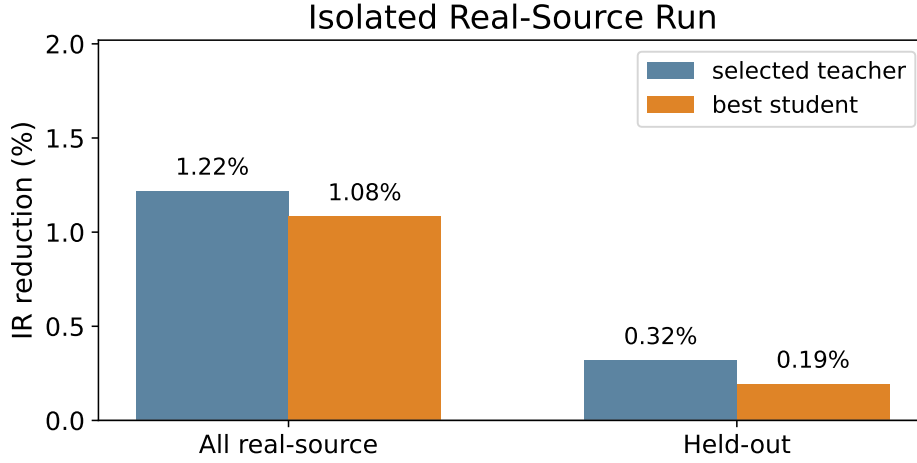


Table 8.4: All-real-source isolated run. The experiment retrains and rewrites only real-source modules in out/real_signal_run_all/, leaving the main reported artifacts unchanged.

Positive reductions mean smaller rewritten IR than never-inline, while negative values mean code growth. The best held-out student is `deep_forest`, with 1559.62 average IR instructions, a 0.61% reduction, and a 7.58-instruction gap to the selected teacher. On the full artifact set, `conservative_forest` is best, reducing all modules by 1.18%. Absolute held-out reductions remain small because many real-source modules leave little profitable local inlining margin under the rewritten-IR objective, and because the rewrite approximation does not recompute features after each accepted decision.

Table 8.3 shows why the learned policies should be read as prototype results rather than production replacements. LLVM -0z remains the stronger native-size baseline, while `knn` and `small_mlp` show that weak generalization can still grow held-out IR.

A three-round BC-Max-style check also fed the previous best student back into the candidate pool. The selected-policy reference stayed at 1.10% held-out IR reduction, and the best student stayed `deep_forest` at 0.61%, so the small policy pool plateaued.

Chapter 9

Limitations

- **Dataset scale.** One of the most important limitations is the scale of the dataset. The run uses 163 compiled C/C++ files and 10394 call-site samples, including 76 public-source files. Broader transfer would still require larger benchmark families such as SPEC, cBench/MiBench, SQLite builds, and wider LLVM test-suite subsets.
- **Held-out scope.** The hold-out split has 1385 samples from 26 unseen modules, but all modules still come from the same source snapshot and generator regime. A stronger future test would use modules from more clearly different sources.
- **Rewrite approximation.** Features are extracted only once in the main run, selected callees are cloned and marked `alwaysinline`, and LLVM `opt` performs the actual inlining. A fuller system would recompute features after each accepted rewrite, closer to LLVM’s online inliner.
- **Metric scope.** Rewritten LLVM IR instruction count is the primary reward. Native `.text` and object-file size remain secondary; linked executable size and runtime are outside the current evaluation.
- **Teacher and MLGO scope.** BC-Max can only select among available candidate policies. The teachers are inspectable proxies rather than production inliner policies, and full MLGO reproduction would require deeper LLVM integration and a much larger corpus.
- **Iterative scope.** The added three-round BC-Max-style check feeds back the best student as a new measured policy, but it does not include the uncertainty-based trajectory exploration used in larger BC-Max systems.
- **Generated-data filtering.** A future generator could add a discriminator that rejects near-duplicate modules and keeps rarer inlining patterns before training.

Chapter 10

Conclusions

The purpose of this thesis was to study how compiler inlining can be optimized using offline methods for learning policies, and to implement a reproducible end-to-end pipeline that works with LLVM IR. The reported run compiles 163 C/C++ sources, evaluates 150 modules with local call sites, evaluates 16 teacher-role heuristics, trains 7 students, and applies learned decisions back to LLVM IR through a call-site-level rewrite.

The main held-out result is measured on a source-level mix of DCMTK, generated modules, and 76 public C/C++ source files. On the 26 held-out modules, the selected-teacher reference reduces average rewritten LLVM IR instruction count from 1569.23 to 1552.04, a 1.10% reduction relative to the never-inline teacher. When it comes to students, the best held-out model by IR size is `deep_forest`, which reaches 1559.62 instructions on average, a 0.61% reduction. On the full artifact set, `conservative_forest` reaches 1961.94 instructions and reduces rewritten IR by 1.18%.

As shown before the conclusion, LLVM `-Oz` is much stronger on the standalone C/C++ files: relative to `no_inline_Oz`, it reduces native `.text` size by 14.91% and object-file size by 28.76%. The learned policies therefore demonstrate the workflow and formulation, not a production-ready replacement for LLVM’s full size pipeline.

The major contribution of this thesis is methodological: it connects source collection, LLVM IR generation, feature extraction, teacher evaluation, behavior-cloning dataset construction, student training, rewrite application, and artifact-level measurement in one reproducible workflow. Stronger results would require integration into LLVM’s actual inliner, larger real benchmark suites, stronger teacher policies, and final linked-binary measurements.

Appendix A

Additional Diagnostics

The generated out/ directory contains the full JSON outputs used for inspection. Figures A.1 to A.5 keep the main diagnostics in portrait layout, with readable labels and values.

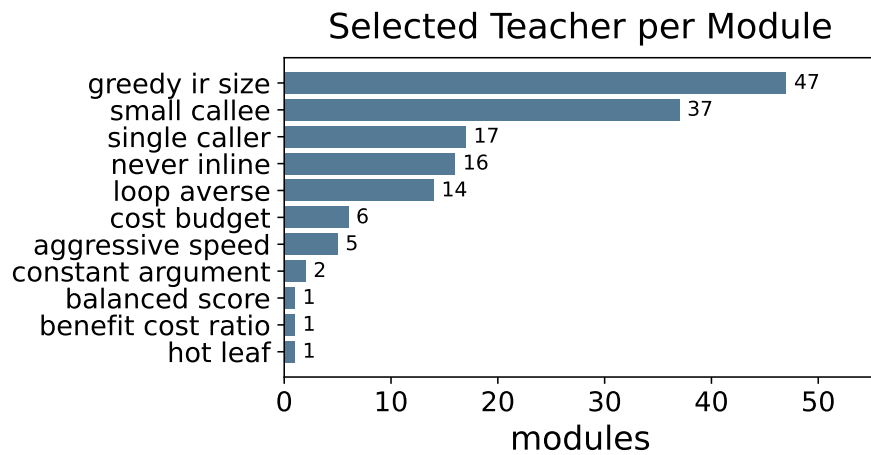


Figure A.1: Selected teacher distribution by module.

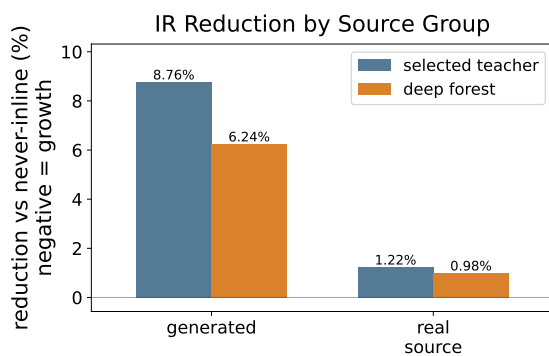


Figure A.2: Teacher and student IR reduction by source group.

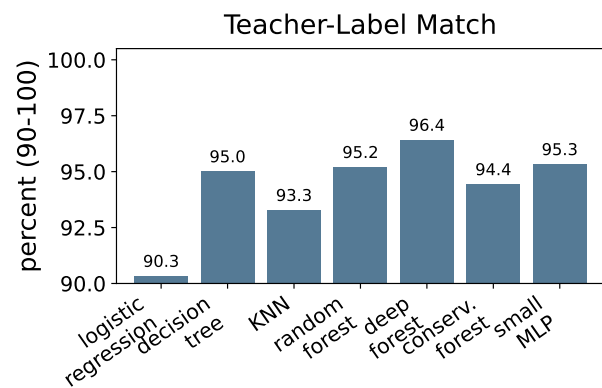


Figure A.3: Held-out teacher-label match for student models.

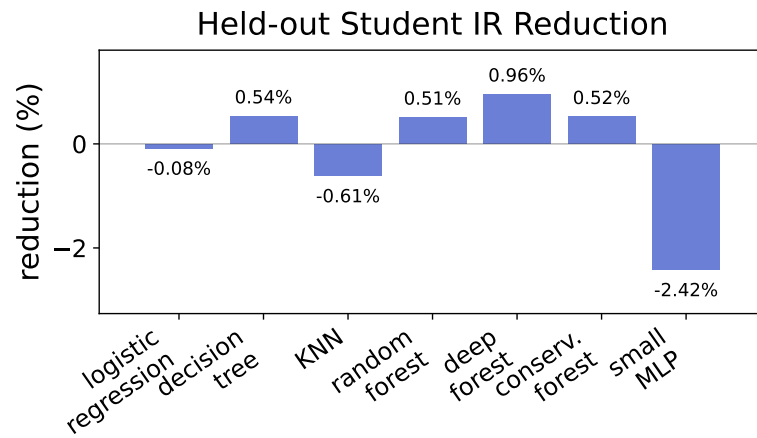


Figure A.4: Held-out student IR reduction.

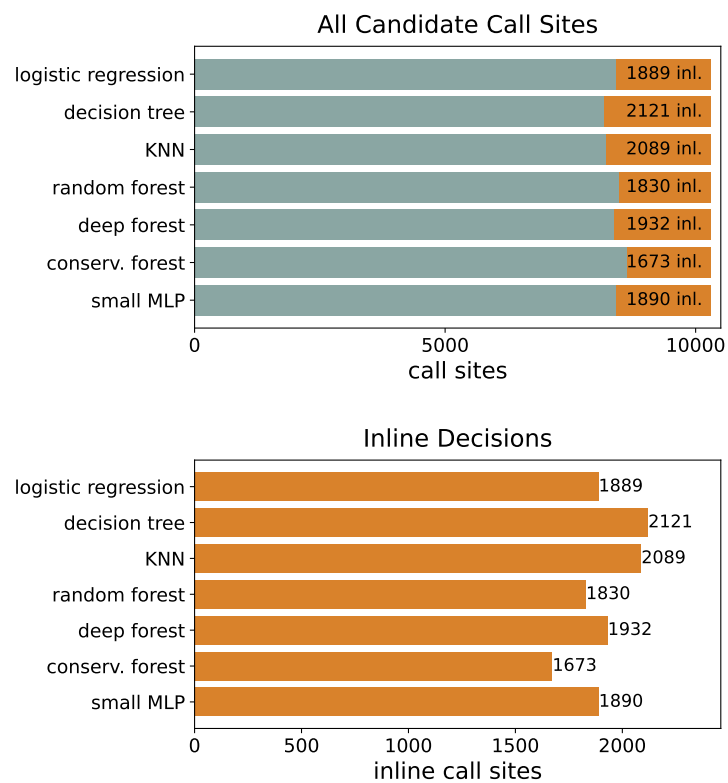


Figure A.5: Student inline/no-inline decisions across candidate call sites.

Bibliography

- [1] M. Godbolt. “Optimizations in C++ Compilers.” In: *ACM Queue* (2019).
- [2] T. Theodoridis, T. Grosser, and Z. Su. “Understanding and Exploiting Optimal Function Inlining.” In: *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ASPLOS ’22. 2022, pp. 977–989.
- [3] M. Trofin et al. “MLGO: A Machine Learning Guided Compiler Optimizations Framework.” In: *arXiv preprint arXiv:2101.04808* (2021).
- [4] G. Fursin et al. “MILEPOST GCC: Machine Learning Enabled Self-tuning Compiler.” In: *International Journal of Parallel Programming* 39.3 (2011), pp. 296–327.
- [5] LLVM Project. *Machine Learning Guided Optimization*.
- [6] T. V. Marinov, A. Agarwal, and M. Trofin. “Offline Imitation Learning from Multiple Baselines with Applications to Compiler Optimization.” In: *arXiv preprint arXiv:2403.19462* (2024).
- [7] K. D. Cooper and L. Torczon. *Engineering a Compiler*. 2nd ed. Morgan Kaufmann, 2011.
- [8] S. S. Muchnick. *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [9] LLVM Project. *LLVM Language Reference Manual*.
- [10] LLVM Project. *LLVM Passes*.
- [11] M. Godbolt. *Inlining - the Ultimate Optimisation*.
- [12] J. Cavazos and M. F. P. O’Boyle. “Automatic Tuning of Inlining Heuristics.” In: *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 2005.
- [13] A. H. Ashouri et al. “MLGOPerf: An ML Guided Inliner to Optimize Performance.” In: *arXiv preprint arXiv:2207.08389* (2022).